

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: " Бинарное дерево "

Выполнил:

Студент группы 24ВВВ3

Макунькин А.М.

Мелюшев М.М.

Принял:

к.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение бинарное дерево поиска.

Лабораторное задание

Задание:

- Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
- Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Результат программы

```
D:\pr\Lab4\Debug\Lab4! X + v
Введите число: 4
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 5
Введите число: 2
Введите число: 1
Введите число: 3
Введите число: 6
Введите число: 5
Введите число: 4
Введите число: 7
Введите число: 8
Введите число: 5
Введите число: -1

=== Дерево построено ===

      9(3)
     /  \
    9(2)  8(6)
   /  \  /  \
  7(5) 6(5) 7(5) 8(6)
 /  \  /  \  /  \
6(3) 5(6) 6(5) 7(5) 8(6) 9(6)
/  \  /  \  /  \  /  \
5(1) 4(3) 5(5) 6(6) 7(7) 8(8) 9(9)
/  \  /  \  /  \  /  \
4(2) 3(4) 4(4) 5(5) 6(6) 7(7) 8(8) 9(9)
/  \  /  \  /  \  /  \  /  \
3(3) 2(3) 3(3) 4(4) 5(5) 6(6) 7(7) 8(8) 9(9)
/  \  /  \  /  \  /  \  /  \
2(2) 1(4) 2(2) 3(3) 4(4) 5(5) 6(6) 7(7) 8(8) 9(9)
/  \  /  \  /  \  /  \  /  \
1(1) 0(0) 1(1) 2(2) 3(3) 4(4) 5(5) 6(6) 7(7) 8(8) 9(9)

=== Задание 1: Поиск элемента ===
Введите число для поиска: 5
Число 5 найдено в дереве!

=== Задание 2: Подсчет вхождений элемента ===
Введите число для подсчета: 5
Число 5 встречается в дереве 4 раз(а).

=== Задание 3: Определение уровня последнего искомого элемента ===
Введите число для определения уровня: 5
Уровень последнего вхождения числа 5: 6

Нажмите Enter для выхода...
```

Рисунок 1 – бинарное дерево

Листинг

Файл 1-Lab3L.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* insertNode(struct Node* root, int data) {
    if (root == NULL) {
        return createNode(data);
    }

    if (data < root->data) {
        root->left = insertNode(root->left, data);
    }
    else {
        root->right = insertNode(root->right, data);
    }

    return root;
}

struct Node* searchNode(struct Node* root, int key) {
    if (root == NULL || root->data == key) {
        return root;
    }

    if (key < root->data) {
        return searchNode(root->left, key);
    }
    else {
        return searchNode(root->right, key);
    }
}

int countOccurrences(struct Node* root, int key) {
    if (root == NULL) {
        return 0;
    }

    int count = 0;

    if (root->data == key) {
        count = 1;
    }

    return count + countOccurrences(root->left, key) + countOccurrences(root->right, key);
}

int searchLevel(struct Node* root, int key, int level) {
    if (root == NULL) {
```

```

        return 0;
    }

    int currentLevel = 0;

    if (root->data == key) {
        currentLevel = level;
    }

    int rightLevel = searchLevel(root->right, key, level + 1);
    if (rightLevel != 0) {
        return rightLevel;
    }

    if (currentLevel != 0) {
        return currentLevel;
    }

    return searchLevel(root->left, key, level + 1);
}

void printTree(struct Node* root, int space, int level) {
    if (root == NULL) {
        return;
    }

    space += 5;

    printTree(root->right, space, level + 1);

    printf("\n");
    for (int i = 5; i < space; i++) {
        printf(" ");
    }
    printf("%d(%d)\n", root->data, level);

    printTree(root->left, space, level + 1);
}

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);

    struct Node* root = NULL;
    int userInput;
    int searchKey;

    printf("=== Построение бинарного дерева поиска ===\n");
    printf("Вводите целые числа. Для окончания ввода введите -1.\n");

    while (1) {
        printf("Введите число: ");
        scanf("%d", &userInput);

        if (userInput == -1) {
            break;
        }

        root = insertNode(root, userInput);
    }

    printf("\n=== Дерево построено ===\n");
    printTree(root, 0, 1);

    printf("\n=== Задание 1: Поиск элемента ===\n");
    printf("Введите число для поиска: ");
    scanf("%d", &searchKey);

    struct Node* searchResult = searchNode(root, searchKey);
    if (searchResult != NULL) {

```

```

        printf("Число %d найдено в дереве!\n", searchKey);
    }
    else {
        printf("Число %d не найдено в дереве.\n", searchKey);
    }

    printf("\n=== Задание 2: Подсчет вхождений элемента ===\n");
    printf("Введите число для подсчета: ");
    scanf("%d", &searchKey);

    int count = countOccurrences(root, searchKey);
    printf("Число %d встречается в дереве %d раз(a).\n", searchKey, count);

    printf("\n=== Задание 3: Определение уровня последнего искомого элемента ===\n");
    printf("Введите число для определения уровня: ");
    scanf("%d", &searchKey);

    int level = searchLevel(root, searchKey, 1);
    if (level != 0) {
        printf("Уровень последнего вхождения числа %d: %d\n", searchKey, level);
    }
    else {
        printf("Число %d не найдено в дереве.\n", searchKey);
    }

    printf("\nНажмите Enter для выхода...");
    getchar();
    getchar();

    return 0;
}

```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №4. В процессе выполнения работы были использованы знания о бинарном дереве.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/LareklAREK/LiOAiIZ-2OURSE-.git>

<https://github.com/Motorrr/LiOAvIZ.git>